

LANGAGE DE MANIPULATION DES DONNÉES (LMD)

➤ INTRODUCTION

➤ COMMANDE SELECT

➤ SELECT-PROJECTION

➤ SELECT-SÉLECTION

➤ SELECT-COMPARAISON À UNE VALEUR

➤ SELECT-COMPARAISON À UNE FOURCHETTE DE VALEUR

➤ SELECT-COMPARAISON À UNE LISTE DE VALEURS

➤ SELECT-COMPARAISON À UN FILTRE

➤ TRI DES TUPLES

➤ CLAUSE GROUP BY

➤ FONCTIONS DE GROUPE

➤ FONCTION COUNT

➤ FONCTION SUM

➤ FONCTION AVG

➤ FONCTIONS MIN ET MAX

➤ CLAUSE HAVING

➤ CLAUSE LIMIT

➤ COMMANDE INSERT

➤ COMMANDE UPDATE

➤ COMMANDE DELETE

Introduction

- ❖ Le **Langage de Manipulation des Données** (LMD ou DML-Data Manipulation Language) est une partie du SQL permettant d'interagir avec les données stockées dans une base de données
- ❖ Les principales commandes LMD sont:
 - **SELECT** pour récupérer les données
 - **INSERT** pour ajouter les données
 - **UPDATE** pour modifier les données
 - **DELETE** pour supprimer les données

Commande SELECT

- ❖ **SELECT** est utilisée pour interroger une base de données et récupérer les données depuis une ou plusieurs tables.
- ❖ Elle permet de réaliser les opérations suivantes:
 - Les projections
 - Les sélections
 - Les tri des tuples
 - Les fonctions et expressions
 - Les requêtes sur les groupes

Commande SELECT

- ❖ La syntaxe globale d'une commande **SELECT** est la suivante:

SELECT ...	}	Partie obligatoire
FROM ...		
WHERE ...	}	Clauses
GROUP BY ...		
HAVING ...		
ORDER BY ...		
LIMIT...		

- ❖ L'ordre des **différentes clauses** est à respecter

SELECT - Projection

❖ En SQL, une **projection** fait référence à l'opération qui consiste à sélectionner certaines colonnes d'une table

❖ La syntaxe est la suivante

- **SELECT** colonne1, colonne2, ... **FROM** nom_table;

❖ **Exemples:**

- **SELECT *** FROM client; (*) signifie que **toutes les colonnes** seront affichées
- **SELECT** nom FROM client; toutes les lignes de la table client sont affichées
- **SELECT** nom, ville FROM client; les colonnes 'nom' et 'ville' seront affichées
- ❌ **SELECT** nom **AS** nom_client FROM client; la colonne 'nom' sera renommée en 'nom_client' avant d'être affichée

SELECT - Projection

- ❖ Si on veut éviter que les lignes issue d'une **projection** ne se répètent pas, on va utiliser la clause **DISTINCT**

- ❖ **Exemple**

- SELECT Type_Stage FROM stage;

Type-Stage
TP
TP
TP
TP
TP
Cours
TP
Démonstration
Démonstration

- SELECT DISTINCT Type_Stage AS nature du stage FROM stage;

nature du stage
TP
Cours
Démonstration

- ❖ **Remarque:** Si on indique **plusieurs noms de tables derrière le FROM**, on obtient un **produit cartésien**

SELECT - Sélection

- ❖ En SQL, une **sélection** est une opération qui permet de spécifier en plus des colonnes, les lignes qui devront s'afficher à l'issue d'une requête. Cela se fait avec la clause **WHERE**
- ❖ La syntaxe est la suivante:
SELECT colonne1, colonne2, ...
FROM nom_table;
WHERE (condition)
- ❖ La clause **WHERE** permet de spécifier quelles sont les lignes à sélectionner dans une table ou dans le produit cartésien de plusieurs tables.
- ❖ WHERE est suivie d'une **expression logique** prenant la valeur **vrai** ou **faux** et qui sera évalué pour chaque ligne. **Seules les lignes pour lesquelles l'expression logique est évalué à vrai sont sélectionnées.**

SELECT - Sélection

- ❖ Les **expressions logiques** peuvent être composées d'une suite de conditions combinées entre elles par les opérateurs **AND, OR, NOT**.
- ❖ Des parenthèses peuvent être utilisées pour imposer une priorité dans l'évaluation de l'expression logique. L'opérateur **NOT** placé devant un prédicat en **inverse le sens**.
- ❖ **Les différentes formes d'expressions logiques:**
 - Comparaison à une valeur,
 - Comparaison à une fourchette de valeurs,
 - Comparaison à une liste de valeurs,
 - Comparaison à un filtre,
 - Test sur l'indétermination d'une valeur,
 - Test ' tous ' ou ' au moins un ',
 - Test existentiel,
 - Test d'unicité

SELECT - Sélection

❖ Quelques cas d'expressions logiques

- ❖ WHERE exp1 = exp2
 - ❖ WHERE exp1 != exp2
 - ❖ WHERE exp1 < exp2
 - ❖ WHERE exp1 > exp2
 - ❖ WHERE exp1 <= exp2
 - ❖ WHERE exp1 >= exp2
 - ❖ WHERE exp1 is NULL
 - ❖ WHERE exp1 is NOT NULL
- ❖ Exp2 peut être une **constante**. Les **chaînes de caractères** doivent apparaître sous forme d'une **séquence de caractères entre apostrophes**.
- ❖ Exp2 peut être une expression obtenue par application d'opérateurs (+, -, x, /) sur des noms d'attributs ou des constantes

SELECT - Comparaison à une valeur

❖ Exemples

- ❖ **SELECT** Num_St, Nom_St **FROM** STAGIAIRE **WHERE** Adr= 'Paris';
- ❖ **SELECT** Num-Stage **FROM** STAGE **WHERE** Nb-jours > 4;
- ❖ **SELECT** Num-Stage **FROM** STAGE **WHERE** (Prix* nb-jours) > 5000;
- ❖ **SELECT** Num-Stage, Libellé-Stage, Type-Stage, Num-Cat
FROM STAGE
WHERE Type-Stage = 'TP ' **AND** Num-Cat = 1;
- ❖ **SELECT** Num-Stage, Libellé-Stage, Type-Stage, Num-Cat
FROM STAGE
WHERE **NOT**(Type-Stage = 'TP ' **OR** Num-Cat = 1);

SELECT - Comparaison à une fourchette de valeurs

❖ La comparaison sur une **fourchette de valeurs** se fait à l'aide de la clause **BETWEEN...AND...**

❖ **Syntaxe:**

```
SELECT ... FROM ... WHERE Cond1 [NOT] BETWEEN val1 AND val2;
```

❖ **Exemple**

```
SELECT Num-Stage, Libellé-Stage, Num-Cat  
FROM STAGE  
WHERE Num-Cat BETWEEN 1 AND 3;
```

SELECT - Comparaison à une liste de valeurs

- ❖ La comparaison sur une **liste de valeurs** se fait à l'aide de la clause **IN**
- ❖ **Syntaxe:**
SELECT ... FROM ... WHERE nom_colonne **[NOT] IN** (liste_valeurs);
- ❖ **Exemple**
SELECT Num-Stage, Libellé-Stage, Num-Cat
FROM STAGE
WHERE Num-Cat **IN** (1,3);
- ❖ **NOT IN** sert à sélectionner les lignes dont l'attribut dans la clause **WHERE** contient une valeur autre que celle de la liste

SELECT - Comparaison à un filtre

❖ La comparaison sur un **filtre** est faite à l'aide la clause **LIKE**. Cette clause **s'utilise uniquement sur les chaines de caractères**

❖ **Syntaxe:**

SELECT ... FROM ... WHERE nom_colonne **[NOT] LIKE** 'Modèle_de_chaine';

❖ Cette clause teste l'égalité de deux chaînes en tenant compte des **caractères jokers** dans la 2ème chaîne.

- « **_** » remplace 1 caractère exactement,
- « **%** » remplace une chaîne de caractères de longueur quelconque.
- **'%objet%'** sélectionne toutes les chaines contenant **objet** à n 'importe quelle place dans la chaîne
- **'%a_'** sélectionne toutes les chaines ayant **a** à l'avant-dernière place
- **'%@%'** sélection toutes les chaines ayant **%** n'importe où dans la chaîne.

SELECT - Comparaison à un filtre

❖ Exemples

- Liste des stagiaires dont les noms commencent par 'Var'
 - ✓ **SELECT** Num-St, Nom-St, Prénom-St
FROM STAGIAIRE
WHERE Nom-St **LIKE** 'Var%';
- Liste des stagiaires dont les noms tiennent sur 8 caractères, commençant par PE puis se terminant par N
 - ✓ **SELECT** Num-St, Nom-St, Prénom-St
FROM STAGIAIRE
WHERE Nom-St **LIKE** 'PE____N';
- Liste des stagiaires dont les noms tiennent sur 3 caractères et se terminant par AR
 - ✓ **SELECT** Num-St, Nom-St, Prénom-St
FROM STAGIAIRE
WHERE Nom-St **LIKE** '_AR%';

SELECT - Tri des tuples

❖ L'ordre dans lequel les lignes d'une table résultat apparaissent est indéterminé. Pour imposer un ordre d'affichage, on utilise la clause

ORDER BY.

❖ **Syntaxe :**

SELECT ... FROM ... ORDER BY colonne1 [DESC|ASC], colonne2 [DESC|ASC],... ;

❖ On peut trier selon la valeur d'un ou de plusieurs attributs.

❖ Les attributs selon lesquels le tri est demandé doivent obligatoirement faire partie de la clause SELECT.

❖ **DESC** donne un tri par ordre décroissant, et **ASC** donne un tri croissant. Par défaut, l'ordre est croissant.

SELECT - Tri des tuples

❖ Le tri se fait d'abord selon le premier attribut, puis les lignes ayant la même valeur pour ce premier attribut sont triées selon le 2ème attribut, etc. Les valeurs nulles sont toujours en tête.

❖ Exemples

▪ Tri par ordre croissant de Num_Stage

✓ `SELECT Num-Stage, Libellé-Stage //`
`FROM STAGE`
`ORDER BY Num-Stage;`

▪ Tri par ordre croissant sur la 1^{ière} colonne

✓ `SELECT Num-Stage, Libellé-Stage // Tri par ordre croissant sur la 1ière colonne`
`FROM STAGE`
`ORDER BY 1;`

▪ Tri par ordre croissant sur Num_Cat et par ordre décroissant sur Num_Stage

✓ `SELECT Num-Stage, Libellé-Stage, Nom-Cat`
`FROM STAGE`
`WHERE NOT(Type-Stage = 'Cours')`
`ORDER BY Num-Cat, Num-Stage DESC;`

Clause GROUP BY

❖ **GROUP BY** est utilisée pour regrouper les lignes ayant des valeurs identiques dans une ou plusieurs colonnes. Cette clause permet d'effectuer des opérations d'agrégation (COUNT, SUM, AVG, etc.)

❖ **Syntaxe :**

SELECT COUNT | SUM | AVG | MIN | MAX

FROM nom_table;

[WHERE condition]

GROUP BY nom_colonne

[HAVING condition]

NOTEZ BIEN:

❖ **En présence d'un GROUP BY**, tout attribut dans la clause SELECT doit figurer dans la clause GROUP BY.

Fonctions de groupes

- ❖ Un **groupe** est un ensemble de tuples d'une table pour lesquels la valeur d'un attribut est constante.
- ❖ Les **fonctions de groupe** effectuent un calcul sur l'ensemble des valeurs d'un attribut pour un groupe de tuples.
- ❖ Il existe 5 fonctions de groupes:
 - COUNT,
 - SUM,
 - AVG,
 - MIN,
 - MAX

Fonctions de groupes

- ❖ Une fonction peut apparaître dans une clause SELECT, elle sera affichée comme un attribut ou une expression **MAIS**, en l'absence de clause GROUP BY, on ne peut mettre ensemble des fonctions de groupe et des attributs dans un même SELECT.
- ❖ Une **fonction de groupe** s'emploie uniquement dans les clauses **SELECT** ou **HAVING**.

Fonction COUNT

❖ **COUNT** permet de compter le nombre de ligne dans un ensemble de résultats qui répondent à des critères spécifiques

❖ **Syntaxe :**

```
SELECT COUNT(* | [DISTINCT] nom_colonne)
FROM nom_table;
[WHERE condition]
[GROUP BY nom_colonne]
[HAVING condition]
```

Fonction COUNT

❖ Exemples

- **SELECT COUNT(*) FROM Employes** // Nombre de lignes dans la table Employes
- **SELECT COUNT(*)** // Nombre d'employés dont le salaire est supérieur à 100000
FROM Employes
WHERE salaire > 100000;
- **SELECT** poste, **COUNT(*)** // Nombre d'employés par poste
FROM Employes
GROUP BY poste;
- **SELECT COUNT(DISTINCT salaire)** // Nombre de valeurs possibles prises par la colonne "salaire"

Fonction SUM

❖ **SUM** est utilisé pour calculer la somme des valeurs d'une colonne numérique dans un ensemble de résultats.

❖ **Syntaxe :**

```
SELECT SUM(colonne)
```

```
FROM nom_table;
```

```
[WHERE condition]
```

```
[GROUP BY]
```

```
[HAVING]
```

Fonction SUM

❖ Exemples

- La somme des salaires dans une table employes
 - ✓ **SELECT SUM(salaire) FROM** Employes;
- La somme des ventes du produit dont le code est 123
 - ✓ **SELECT SUM(montant)**
FROM Ventes
WHERE produit_id = 123;
- La somme des salaires par poste
 - ✓ **SELECT poste, SUM(salaire)**
FROM Employes
GROUP BY poste;

Fonction AVG

❖ **AVG** est utilisé pour calculer la moyenne des valeurs d'une colonne numérique dans un ensemble de résultats

❖ **Syntaxe :**

```
SELECT AVG(nom_colonne)
FROM nom_table;
[WHERE condition]
[GROUP BY]
[HAVING]
```


Fonction AVG

❖ Exemples

- La somme des salaires dans une table employes
 - ✓ **SELECT** **AVG**(salaire) **FROM** Employes;
- La moyenne des ventes du produit dont le code est 123
 - ✓ **SELECT** **AVG**(montant)
FROM Ventes
WHERE produit_id = 123;
- La moyenne des salaires par poste
 - ✓ **SELECT** poste, **AVG**(salaire)
FROM Employes
GROUP BY poste;

Fonctions MIN et MAX

❖ **MIN** et **MAX** sont utilisés pour obtenir les valeurs minimale et maximale respectivement d'une colonne spécifiée dans un ensemble de résultats.

❖ **Syntaxe :**

```
SELECT MIN(nom_colonne) | MAX (nom_colonne)
FROM nom_table;
[WHERE condition]
[GROUP BY]
[HAVING (condition)]
```

Fonctions MIN et MAX

❖ Exemples

- Les salaire minimum et maximum dans la table Employes
SELECT MIN(salaire), MAX(salaire) **FROM** Employes;
- La date la plus ancienne et la plus récente dans une table Commandes
SELECT MIN(date_commande), MAX(date_commande) **FROM** Commandes;
- Les salaires minimum et maximum des employés par département
SELECT departement, MIN(salaire), MAX(salaire)
FROM Employes
GROUP BY departement;

Clause HAVING

❖ **HAVING** est utilisé pour filtrer les résultats d'une requête après que les données aient été regroupées par la clause GROUP BY

❖ **Syntaxe :**

SELECT SUM | AVG | COUNT | MIN | MAX

FROM nom_table;

[**WHERE** condition]

GROUP BY

HAVING (condition)

Clause HAVING

❖ Exemples

- Compter les départements ayant plus de 5 employés

```
SELECT departement, COUNT(*)
```

```
FROM Employes
```

```
GROUP BY departement
```

```
HAVING COUNT(*) > 5;
```

- Les codes de produits dont la somme des ventes est supérieure à 10000

```
SELECT produit_id, SUM(montant)
```

```
FROM Ventes
```

```
GROUP BY produit_id
```

```
HAVING SUM(montant) > 10000;
```

Clause LIMIT

❖ **LIMIT** est utilisé pour restreindre le nombre de résultats renvoyés par une requête. Cette clause est particulièrement utile lorsqu'on souhaite récupérer un sous ensemble de résultats

❖ **Syntaxe :**

SELECT

FROM nom_table;

[WHERE condition]

[GROUP BY]

[HAVING (condition)]

LIMIT valeur **[OFFSET valeur]**

Clause LIMIT

❖ Exemples

- Les 5 premières lignes de la table Employes

SELECT * FROM Employes **LIMIT 5;**

- Les 10 produits ayant les prix les plus élevés

SELECT *
FROM Produits
ORDER BY prix **DESC**
LIMIT 10;

- Les 10 employés suivants (du 6ième au 15ième)

SELECT *
FROM Employes
LIMIT 5 OFFSET 10;

Commande INSERT

❖ **INSERT** est utilisé pour ajouter de nouvelles lignes dans une table. Elle présente deux syntaxe, selon que l'on souhaite insérer une ligne ou plus d'une ligne

❖ **Insertion d'une ligne**

```
INSERT INTO nom_table (colonne1, colonne2, colonne3)  
VALUES (valeur1, valeur2, valeur3);
```

❖ **Insertion de plus d'une ligne**

```
INSERT INTO nom_table (colonne1, colonne2, colonne3)  
VALUES (valeur1a, valeur2a, valeur3a),  
        (valeur1b, valeur2b, valeur3b),  
        (valeur1c, valeur2c, valeur3c);
```


Commande INSERT

❖ Exemples

- Insérer **une ligne** dans la table **Employes**

INSERT INTO Employes (nom, prenom, salaire)

VALUES ('Dupont', 'Jean', 3000);

- Insérer **3 lignes** dans la table **Produits**

INSERT INTO Produits (nom, prix, quantite)

VALUES ('Produit A', 10.50, 100),

 ('Produit B', 20.00, 200),

 ('Produit C', 15.75, 150);

- Insérer **sans spécifier de colonnes**

INSERT INTO Employes

VALUES (NULL, 'Martin', 'Sophie', 2500);

Dans cet exemple, **NULL** est utilisé pour indiquer que l'id sera auto-incrémenté

Commande UPDATE

❖ **UPDATE** est utilisé pour modifier des enregistrements existants dans une table.

❖ **Syntaxe :**

UPDATE nom_table

. **SET** colonne1 = valeur1, colonne2 = valeur2, ...

WHERE condition;

- **SET** est utilisé pour spécifier les colonnes à mettre à jour et à leurs nouvelles valeurs
- **WHERE** indiquent quelles lignes doivent être mise à jour. En cas d'omission de cette clause, ce sont toutes les lignes de la table qui sont mise à jour

Commande UPDATE

❖ Exemples

- **Modifier le salaire de l'employé dont l'id est 1**

UPDATE Employes

SET salaire = 3500

WHERE id = 1;

- **Modifier le nom et le salaire d'un employé dont l'id est 2**

UPDATE Employes

SET nom = 'Durand', salaire = 4000

WHERE id = 2;

- **Augmenter le salaire de tous les employés du département de vente**

UPDATE Employes

SET salaire = salaire * 1.10

WHERE departement = 'Vente';

Commande DELETE

❖ **DELETE** est utilisé pour supprimer des enregistrements d'une table

❖ **Syntaxe :**

DELETE FROM nom_table

WHERE condition;

- **WHERE** indiquent quelles lignes doivent être supprimées. En cas d'omission de cette clause, ce sont toutes les lignes de la table qui sont supprimées

❖ **NB:** Avant d'effectuer des suppressions massives, il est judicieux de faire une sauvegarde de données, surtout lorsqu'on n'est pas sûr de la commande

Commande DELETE

❖ Exemples

- Supprimer l'employé dont l'id est 1

DELETE FROM Employes

WHERE id = 1;

- Supprimer tous les employés du département de vente

DELETE FROM Employes

WHERE departement = 'Vente';

- Supprimer l'employé Dupont Jean

DELETE FROM Employes

WHERE nom = 'Dupont' AND prenom = 'Jean';

- Supprimer tous les employés d'une table

DELETE FROM Employes;